

Workshop Module Build Proposal

Four screens that let a learner find the most profitable combination of two business decisions — written for the people building it, not for mathematicians.

The whole module in one sentence. The learner enters a few numbers about a business, the app draws profit as a landscape, and the learner walks that landscape to the highest point — with the app showing, at every step, which direction is uphill and by how much.

MODULE	SCREENS	MATHS NEEDED TO BUILD	TEST DATA
Multivariable modeling workshop	4	Arithmetic only	Included, page 8

What we are building, in plain terms

No calculus vocabulary is required to build this module. Everything the app does reduces to evaluating a few arithmetic expressions and drawing the results.

The problem the learner is solving

A shop sells two things — say puffs and tea. It controls two numbers: **how many puffs** it aims to sell in a day, and **how many teas**. Every pair of those two numbers produces a profit. Sell too few, you lose sales. Sell too many, you have to discount, and the shared oven and counter get congested, so costs rise.

So profit depends on *two* numbers at once. There is exactly one combination that makes the most money, and the shop is almost never sitting on it. The module's job is to find that combination and, more importantly, to **show** it.

THE MENTAL MODEL TO HOLD WHILE BUILDING

Profit is a landscape.
The two inputs are your position on the map.
The best combination is the top of the hill.

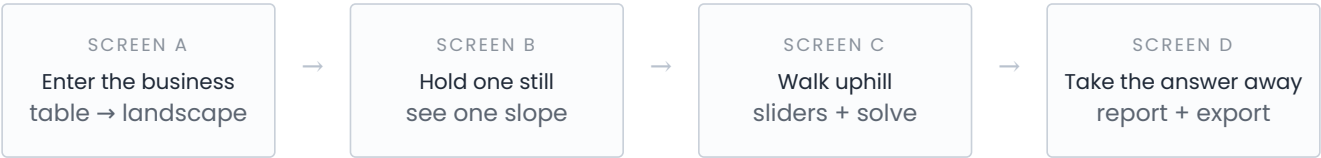
Every screen in this document exists to answer one of three questions: where am I standing, which way is uphill, and how far to the top.

The three things the app computes

NAME IN THE UI	WHAT IT IS	WHAT IT MEANS TO THE LEARNER
Profit	One number, from the two inputs	"If I sell this much of each, I make this much."
Slope (two of them)	One number per input	"If I sell one more puff and change nothing else, my profit moves by ₹X." One slope for puffs, one for tea.
Best combination	One pair of inputs	"This is the top of the hill — both slopes are zero here."

Why "slope" is the whole product. Any calculator can print the best answer. What makes this a teaching tool is that the learner can see the slope at wherever they currently stand — a negative slope on puffs means every extra puff is *losing* money — and can then walk uphill themselves. Build the slope readouts before you build the solver.

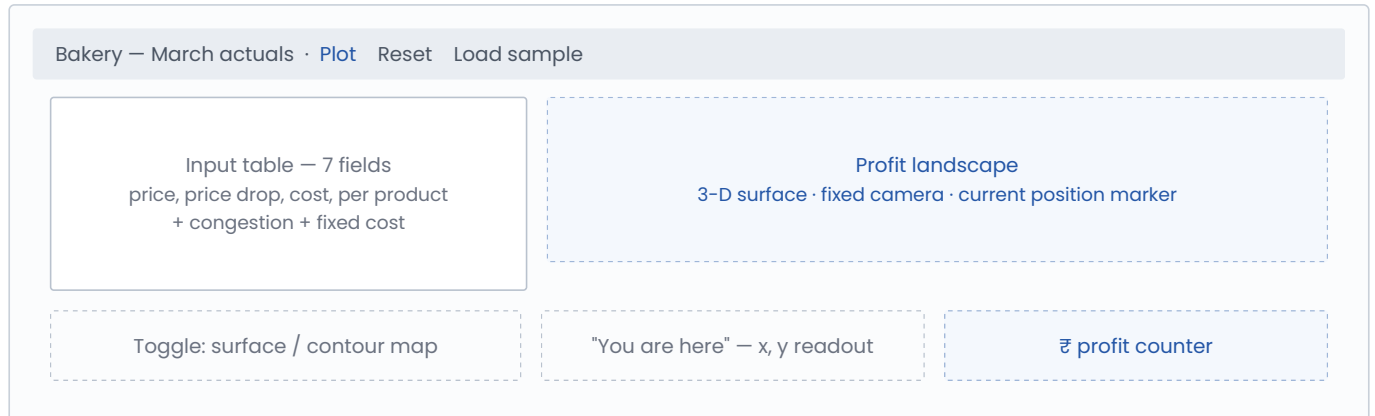
The learner's path through the four screens



Screens B and C are where the learning happens; A and D are input and output. If scope has to be cut, cut depth from A and D — never from B.

Enter the business

The learner types seven numbers about a shop. The app turns them into a picture. Nothing mathematical is shown yet.



Fields on this screen

FIELD	SAMPLE VALUE	PLAIN MEANING SHOWN AS HELPER TEXT
Product 1 — starting price	30	Price if you sold almost none
Product 1 — price drop per unit	0.05	How much you must discount to sell one more
Product 1 — cost per unit	12.50	What it costs you to make one
Product 2 — starting price	16	Same three fields, second product
Product 2 — price drop per unit	0.02	
Product 2 — cost per unit	4.50	
Congestion cost	0.01	Extra cost when both products are busy at once — shared oven, shared counter, shared staff
Fixed cost per day	1000	Rent, power, salaries
Current position (x, y)	200, 150	What the shop actually sells today

Behaviour

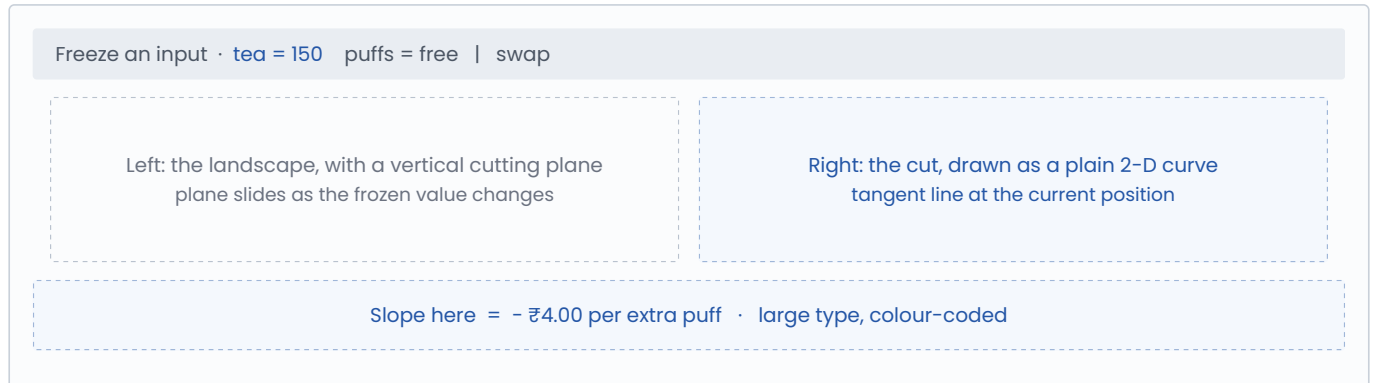
- **Plot** renders the surface and drops a marker at the current position. Both must appear together — the marker is the emotional hook, not the surface.
- **Contour toggle** switches to a top-down map with same-profit rings. This becomes the default view from Screen C onward, so build it here.
- **Camera is fixed.** No auto-rotate, no free orbit. Allow at most a small drag with snap-back.
- **Load sample** populates the bakery dataset in one tap. Presenters will use this constantly; it must never require typing.

Acceptance criteria

- ☐ Sample data loads and plots in under 1 second on a mid-range Android device
- ☐ Marker sits visibly below the peak with the sample data
- ☐ Axes are labelled in words and units — "puffs per day", never "x"
- ☐ Contour map uses one colour ramped light-to-dark, 6 to 8 rings, not a rainbow
- ☐ Profit counter appears in the same screen corner used on every other screen

Hold one input still

This is the screen the whole module exists for. It is also the simplest thing on the list to build, and the easiest to get subtly wrong.



What it does

The learner freezes one input — tea at 150 cups — and only the other one can move. On the left, a flat plane slices through the landscape. On the right, the outline of that cut is drawn as an ordinary 2-D curve. The two panels are the same thing viewed two ways, and they must be visibly linked.

At the learner's current position on that curve, draw the tangent line and print its slope in rupees. That number is the answer to "what does one more puff do to my profit right now" — and it is the single most valuable output in the product.

Behaviour

- **The two panels animate together.** Drag the frozen value and the plane slides on the left while the curve redraws on the right, in the same frame. If they lag apart, the connection is lost and so is the lesson.
- **Swap** freezes the other input instead — same layout, other slope. Both directions must work; presenters use both within thirty seconds of each other.
- **Colour the slope readout:** green when positive, amber when negative. Never red — negative here means "reduce this", not "error".
- **Sign in words, not symbols.** Print "each extra puff loses you ₹4.00", not "-4.00". The minus sign alone does not land with non-mathematical users, and this module's whole audience is non-mathematical.

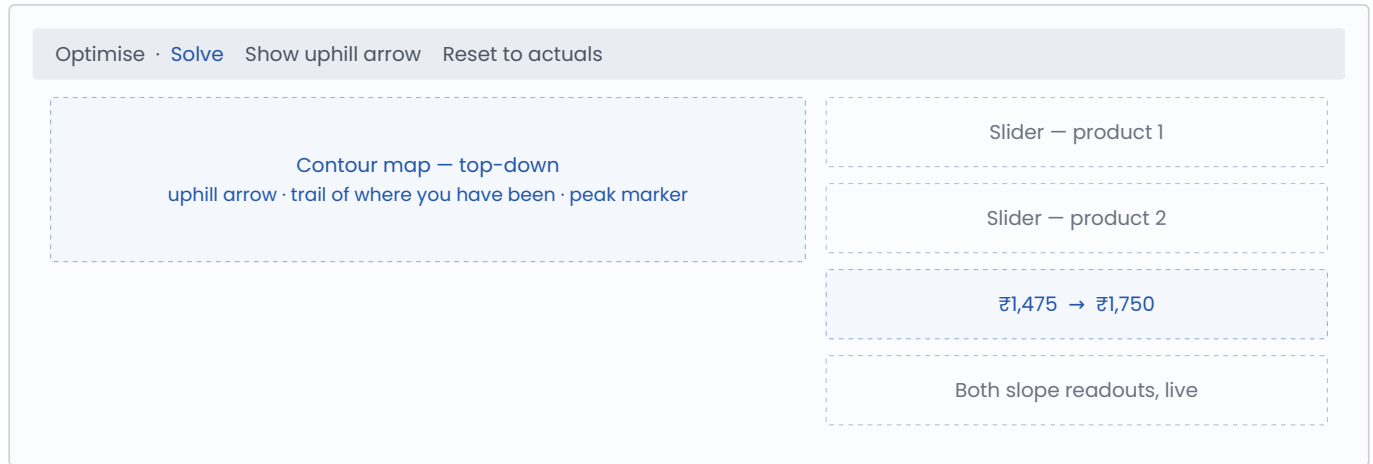
The one thing that cannot ship broken. If the plane-slides-and-curve-redraws animation stutters, or the two panels drift out of sync, the module fails at its only real job. Budget for this screen first and test it on the lowest-spec device you support before anything else is polished.

Acceptance criteria

- ☐ Plane and curve stay in sync at 30fps or better while dragging on the lowest-spec target device
- ☐ Slope readout updates live during the drag, not only when it is released
- ☐ Slope is stated as a sentence in rupees, not as a bare signed number
- ☐ Swapping the frozen input keeps the same layout and the same corner positions
- ☐ With the sample data at 200 puffs and 150 teas, the puff slope reads -₹4.00 and the tea slope +₹3.50

Walk uphill, then solve

The learner moves both inputs by hand, watches the profit climb, and then asks the app to jump straight to the top.



Behaviour

- **Two sliders, one live profit counter.** The counter must update on every frame of a drag. Non-mathematical users track the counter long after they have stopped reading the map.
- **The uphill arrow** is drawn from the current position, pointing in the direction profit increases fastest. Its length should scale with how steep the ground is, so it visibly shrinks to nothing as the learner nears the top. That shrinking is the concept, rendered.
- **Trail:** leave a faint breadcrumb line of positions visited. It lets a presenter show the journey in one glance and costs almost nothing to implement.
- **Solve** animates the marker to the best combination rather than teleporting it, and prints the before/after profit pair.
- **Reset to actuals** must be one tap. Presenters run this demo repeatedly and cannot be re-entering data between rounds.

Acceptance criteria

- ☐ Profit counter updates continuously during a slider drag with no visible stutter
- ☐ Uphill arrow shrinks toward zero length as the marker approaches the best combination
- ☐ Solve lands on 150 and 250 with the sample data, showing ₹1,475 → ₹1,750
- ☐ Reset restores the original position in one tap, sliders included
- ☐ Slider step sizes produce round numbers — no 147.3 puffs

Scope note. Solve is the least important item on this page even though it looks like the headline feature. If it slips, a presenter can compute the answer by hand in thirty seconds. The sliders and the live counter cannot be worked around.

The output the learner keeps

One page, exportable. This is the artefact that leaves the session, and the strongest thing we own commercially.

Report · [Export PDF](#) · Share

Recommendation, in one plain sentence

Best combination

Value of one more unit, each product

How much being off by 10 or 50 costs

Break-even ring

Working shown — collapsed by default, expandable

Contents

BLOCK	SAMPLE OUTPUT	NOTES FOR IMPLEMENTATION
Recommendation sentence	"Sell 150 puffs and 250 teas instead of 200 and 150 — about ₹8,250 more per month."	Template string; round to the nearest ₹50 for the monthly figure
Best combination	150 · 250	Round to whole units
Value of one more unit	−₹4.00 puffs · +₹3.50 tea	Computed at the <i>current</i> position, not at the peak
Cost of being off	10 units off $\approx$ ₹5/day · 50 off $\approx$ ₹125/day	Two fixed offsets is enough; no need for a full band
Break-even ring	The contour where profit = 0	Same contour renderer, one specific level
Working shown	The two equations and their solution	Collapsed by default. Learners who want it will tap; showing it by default scares the rest

Acceptance criteria

- ☐ Exports as a single-page PDF that is legible when printed in black and white
- ☐ Recommendation sentence contains no mathematical notation at all
- ☐ Working section is collapsed on first open, and its state is remembered per learner
- ☐ Report regenerates from any saved dataset without re-running the workshop

Everything the app computes

This is the complete mathematical content of the module. It is six expressions. No libraries, no solver, no calculus at runtime.

Step 1 — turn the seven inputs into six coefficients

```
A = price1 - cost1      // 30 - 12.5 = 17.5
B = priceDrop1          // 0.05
C = price2 - cost2      // 16 - 4.5 = 11.5
D = priceDrop2          // 0.02
E = congestion           // 0.01
F = fixedCost           // 1000
```

Step 2 — profit at any position (drives the surface, the contours, the counter)

```
profit(x, y) = A*x - B*x*x + C*y - D*y*y - E*x*y - F
```

For the surface and the contour map, evaluate this over a grid — 80×80 is ample. For the sample data use x from 0 to 400 and y from 0 to 500.

Step 3 — the two slopes (drives Screen B, the arrow, the report)

```
slopeX(x, y) = A - 2*B*x - E*y    // what one more unit of product 1 does
slopeY(x, y) = C - 2*D*y - E*x    // what one more unit of product 2 does
```

The uphill arrow on Screen C points along $(\text{slopeX}, \text{slopeY})$; its length is proportional to $\sqrt{(\text{slopeX}^2 + \text{slopeY}^2)}$, which naturally falls to zero at the top.

Step 4 — the best combination (drives Solve)

Both slopes are zero at the top. That is two linear equations, and the solution is closed-form — no numerical solver is needed.

```
den = 4*B*D - E*E
bestX = (2*D*A - E*C) / den
bestY = (2*B*C - E*A) / den
```

Guard against $\text{den} \leq 0$. If the congestion value is large relative to the two price-drop values, the landscape has no single peak and the answer is meaningless. Validate on input and show "these numbers don't describe a business with a best combination — try a smaller congestion value" rather than rendering a broken surface. With the sample data $\text{den} = 0.0039$.

Step 5 — the report numbers

```
bestProfit = profit(bestX, bestY)
currentProfit = profit(currentX, currentY)
monthlyGain = (bestProfit - currentProfit) * 30
costOfBeingOff(n) = B * n * n    // n units away from best, per day
```

Data shape to store per saved model

```
{ name, price1, priceDrop1, cost1, price2, priceDrop2, cost2,
  congestion, fixedCost, currentX, currentY,
  labels: { product1, product2, unit1, unit2, currency } }
```

Everything else is derived. Never store computed values — recompute on load, so a corrected input can never disagree with a stale saved answer.

Validation and edge cases

CONDITION	BEHAVIOUR	MESSAGE TO SHOW
$\text{den} \leq 0$	Block plotting	"These numbers don't describe a business with a single best combination — try a smaller congestion value."
Congestion set to 0	Allow	Valid, but note in the UI that the two products no longer affect each other — the landscape becomes two independent problems
Price below cost	Allow, warn	"You lose money on every unit of this product." The landscape is still correct and instructive
Best combination is negative	Clamp to 0, warn	"With these numbers the best you can do is stop selling this product."
Current position far off-grid	Expand the grid	Silent — recompute the plot range so the marker is always visible
Any field left blank	Block plotting	Inline field error; never plot a partial model

Rounding rule throughout: units to whole numbers, rupees to two decimals on slopes and whole rupees everywhere else. The monthly figure in the report rounds to the nearest ₹50 — precision there reads as false confidence.

Build order and test data

Ranked by how much of the learning experience collapses without each item — not by build difficulty.

#	ITEM	PRIORITY	IF IT IS MISSING
1	Freeze + slice, Screen B	Must have	The module has no teaching value. Everything else is a calculator with a chart.
2	Slope readouts in rupees	Must have	Learners see a shape but never learn to read it
3	Contour map	Must have	No usable view for walking uphill; the 3-D surface alone is unreadable
4	Sliders + live counter	Must have	The learner never feels the levers are theirs
5	Sample-data loader	Must have	No demo is possible; every session starts with typing
6	3-D surface view	Should have	Loses some impact on first reveal, no loss of substance
7	Solve button	Should have	Nothing on the day — the answer can be computed by hand
8	Report + PDF export	Should have	Nothing during a session, but it is our strongest commercial asset
9	Breadcrumb trail, uphill arrow animation	Nice to have	Presentation polish
10	Learner-entered custom business	Nice to have	Biggest driver of repeat usage — schedule it for the release after launch

The counter-intuitive one. Solve — the button that looks like the product — is seventh. Screen B's freeze-and-slice, which is barely more than drawing a line through a grid, is first. Please do not let the ordering drift back toward what looks impressive in a screenshot.

Verification dataset — the numbers your build must reproduce

Load the sample bakery data and check every row. All values are exact.

CHECK	EXPECTED	WHERE IT SHOWS IN THE UI
Coefficient den	0.0039	Internal — validation guard
profit(200, 150)	₹1,475	Screen A counter, at the starting position
slopeX(200, 150)	−₹4.00	Screen B, puffs frozen out
slopeY(200, 150)	+₹3.50	Screen B after swap
bestX, bestY	150 · 250	Screen C, after Solve
profit(150, 250)	₹1,750	Screen C counter at the peak
slopeX and slopeY at the peak	0 · 0	Screen C — the arrow must vanish
monthlyGain	₹8,250	Screen D recommendation sentence
costOfBeingOff(10) / (50)	₹5 · ₹125 per day	Screen D sensitivity block

Three questions we need answered back

1. **Lowest-spec target device** — Screen B's linked animation has to be tested there first, and it determines whether the 3-D surface stays in scope at all.
2. **Existing chart library in the app** — if one is already bundled, the contour renderer should be built on it rather than adding a second dependency for one module.
3. **Offline requirement** — sessions are often run in rooms with no usable wifi. If the module must work offline, everything above already does, but the report export needs to render locally rather than server-side.

Glossary — the words that will come up in review

The teaching side of the team uses standard mathematical names for the same things this document describes in plain language. Both columns mean exactly the same thing.

THEY WILL SAY	THIS DOCUMENT SAYS	IN THE CODE
Multivariable function	Profit from two inputs	<code>profit(x, y)</code>
Surface	The landscape	The 3-D plot
Level curve / contour	Same-profit ring	One level of the contour map
Partial derivative	Slope — what one more unit does	<code>slopeX, slopeY</code>
Gradient	The uphill arrow	<code>(slopeX, slopeY)</code>
Stationary point / optimum	Best combination, top of the hill	<code>bestX, bestY</code>
Second-order condition	Confirms the flat point is a peak, not a dip	The <code>den > 0</code> guard

One thing worth agreeing on before build starts. The left column should never appear in the learner-facing UI. Every label, tooltip and message uses the middle column. That single rule is most of what makes this module feel different from every other maths app in the category.